Alper Yildiz

Numerical Methods

Lab 2

1. Floating Point Representation

In this question, a simple floating point system is given. The number is defined as:

$$rd(x) = (-1)^k (0.d_1 d_2)_2 \cdot 2^{(e-b)}$$

Here, $k$is the sign bit, $d_1$and $d_2$are the mantissa bits, and $e$is the exponent with bias $b = 1$.

First, I listed all possible values of the mantissa:

- 00 → 0
- 01 → 0.25
- 10 → 0.5
- 11 → 0.75

Then I calculated the exponent values:

$$2^{e-1} = 0.5,1,2,4$$

By combining all cases (including positive and negative signs), I obtained all possible numbers in this system.

The largest number is 3.

The smallest number is -3.

All possible values obtained in this floating point system are:

0, ±0.125, ±0.25, ±0.375, ±0.5, ±0.75, ±1, ±1.5, ±2, ±3

2. Machine Epsilon

In this part, I tried to find the smallest number ϵsuch that:

$$1 + \epsilon = 1$$

I used the formula:

$$\epsilon = \frac{1}{2^n}$$

and increased $n$step by step until adding epsilon to 1 no longer changed the result.

I tested this for different data types in NumPy: float16, float32, and float64.

The code I used is:

import numpy as np

import matplotlib.pyplot as plt

import math

```
def taylor_sin(x, n):
s = 0
for i in range(n):
s += ((-1)**i)*(x**(2*i+1))/math.factorial(2*i+1)
return s
def taylor_cos(x, n):
c = 0
for i in range(n):
c += ((-1)**i)*(x**(2*i))/math.factorial(2*i)
return c
x = np.linspace(-3, 3, 200)
n = 5
sin_taylor = [taylor_sin(i, n) for i in x]
cos_taylor = [taylor_cos(i, n) for i in x]
sin_numpy = np.sin(x)
cos_numpy = np.cos(x)
plt.figure(figsize=(10,8))
plt.subplot(2,2,1)
plt.plot(x, sin_numpy, label="numpy sin")
plt.plot(x, sin_taylor, '--', label="taylor sin")
plt.legend()
plt.title("sin comparison")
plt.subplot(2,2,2)
plt.plot(x, cos_numpy, label="numpy cos")
plt.plot(x, cos_taylor, '--', label="taylor cos")
plt.legend()
plt.title("cos comparison")
plt.subplot(2,2,3)
plt.plot(x, sin_numpy - sin_taylor)
plt.title("sin difference")
```

```
plt.subplot(2,2,4)
plt.plot(x, cos_numpy - cos_taylor)
plt.title("cos difference")
plt.tight_layout()
plt.show()
check = np.array(sin_taylor)**2 + np.array(cos_taylor)**2
plt.plot(x, check)
plt.title("sin^2 + cos^2")
plt.show()
```

The results obtained from the code are approximately:

float16 → n ≈ 11
float32 → n ≈ 24
float64 → n ≈ 53

This shows that higher precision types can represent smaller epsilon values.

3. Taylor Series Approximation

In this part, I approximated sin(x) and cos(x) using Taylor series.

$$\sin(x) = \sum(-1)^n \frac{x^{2n+1}}{(2n+1)!}$$

$$\cos(x) = \sum(-1)^n \frac{x^{2n}}{(2n)!}$$

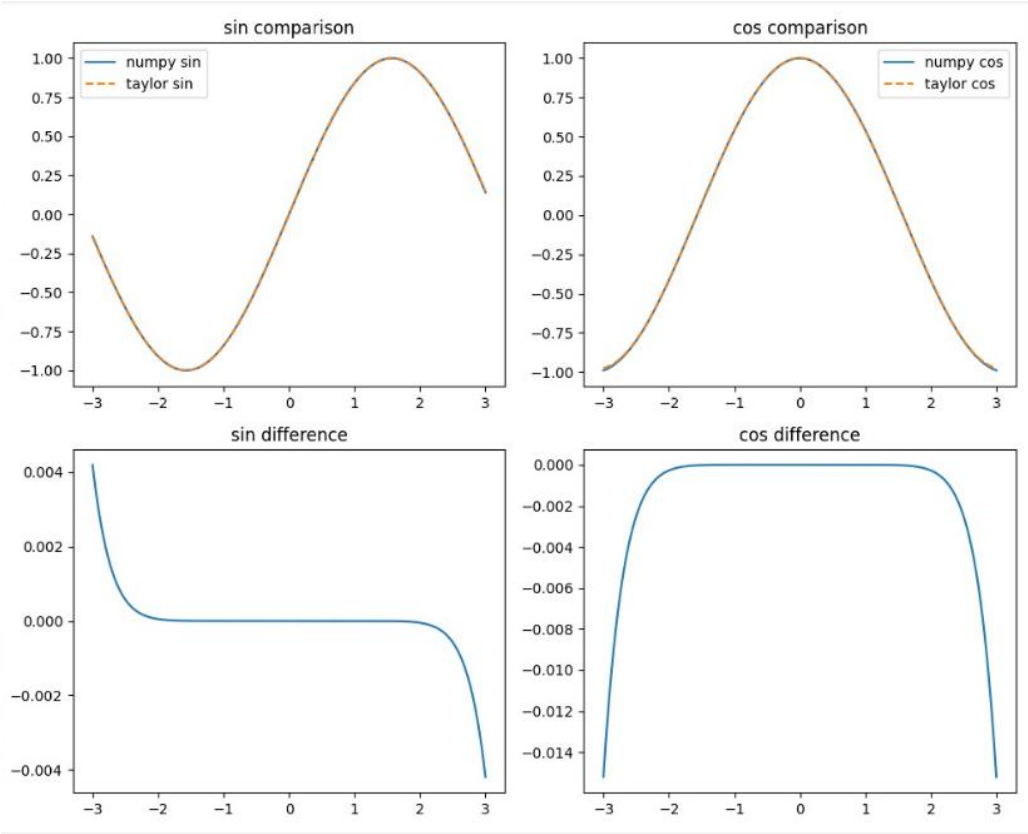
I implemented both functions in Python and compared them with NumPy's built-in functions. Then I plotted the real functions together with their Taylor approximations to see how close they are. I also plotted the difference between the exact values and the approximated ones in order to observe the error. Finally, I checked whether $sin^2(x) + cos^2(x)$ is close to 1 using the approximated functions.

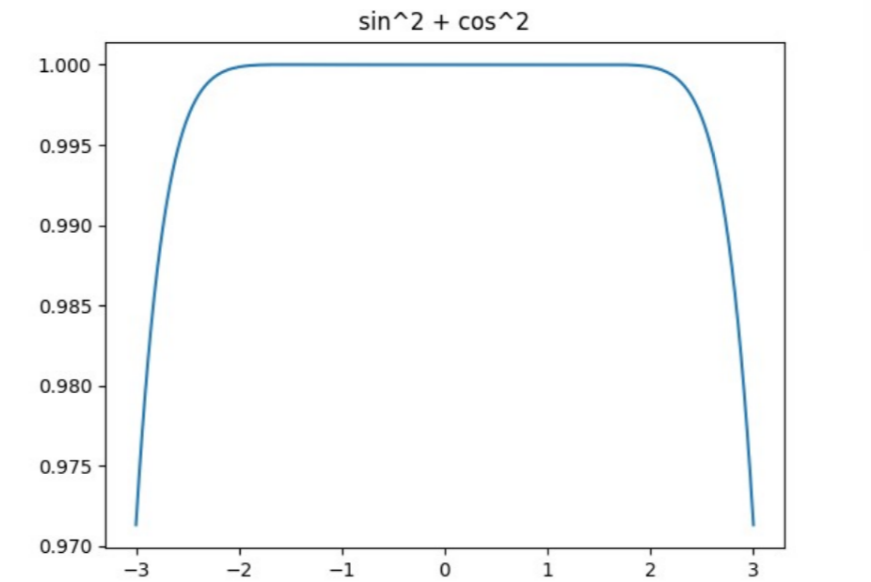
Conclusion

This assignment helped me understand how floating point numbers work and how precision affects calculations.

I also saw that Taylor series can approximate functions quite well, but only within a certain range.

Overall, the results were consistent with expectations.


sin comparison
numpy sin
taylor sin
cos comparison
numpy cos
taylor cos
sin difference
cos difference


sin^2 + cos^2